AI DEVELOPMENT PROMPTS

Model Used: Sonnet 5 (Task 1-4), Opus 5 (Task 5)
Effort: High

*Note: Not all prompts used throughout the project are included, but prompts that will display how I use AI to help develop project code are shown below.

Intro / Project Setup

I'm building the Frontend Developer Assessment. We will be using Angular 19, NgRx, RxJS, and Angular Material. I'll give you the objectives as we go;

Things to note during this project development:
1. Ask questions when there uncertainty in my requests.
2. If there's a better coding convention\standard than the one I suggest let me know.
3.Give me a plan first, folder structure, NgRx design before writing any code, so I can review it.

Task 2 — Customer Management Page

We will now be creating a customer list inside this component.
In @src/app/features/customers/customer-form.component.html
add a material table that's datasource matches the customer model in
@src/app/core/models/customer.model.ts#1-21

Column headers for the table should be Customer, Address, University and Actions. It needs to be sortable.
Also include a searchbar to the table that searches on name, city and suburb.
For actions column add mat-icons for the following actions:

View Quote, Edit Customer and Delete Customer and create empty function for each in @src/app/features/customers/customer-list.component.ts

For the styling match Angular Material's default theming for now, but keep the table and form visually consistent.

I'll do a branding/color pass afterward once functionality is confirmed.

Task 3 — Quote Management Page

in @src/app/features/quotes/quote-list.component.html add a mat-table same as in customer-list that will be showing the quotes of the customer.
Table needs to follow this datasource structure @src/app/core/models/quote.model.ts#3-10

Table columns are :
Quote ID, Customer, Amount, Status, Created and Actions.
Actions are delete and edit for a quote. Also create dummy function and use icons.

From the Customer Management table, in the view quotes action, adjust its onclick function to navigates

to the Quote Management page pre-filtered to that customer's quotes.
It needs to be reactive, not just as a one-time snapshot, so it stays correct if the underlying data changes.

Keep the two tables visually consistent with each other (same header/row styling approach) since a reviewer will compare them side by side.

Task 4 — Add, Edit, Delete via NgRx

Wire up add/edit/delete for both customers and quotes through NgRx Store:

actions, reducers, and effects, not direct service calls from components.

Every add/edit/delete should update the store and reflect immediately in both tables.

State management:
Use @ngrx/entity for the reducers.

Effects should be async simulate a real API integration.

If a customer is deleted, delete their quotes too.

Add a confirmation dialog before any delete (customer or quote) using mat-dialog. cancelling must be a no-op.

Walk me through where in Redux DevTools I should see the add/update/delete actions fire,

so I can verify this is genuinely going through the store and not just mutating local component state.

TASK 5

Step 1 — Architecture & API

This is Task 5 of the assessment,

the enrichment panel that will be inside the add/edit customer form will now be designed.

It needs to predict nationality from surname with the Nationalize API, let the user confirm or pick a different country (countries.dev),

then search universities in that country (hipolabs).

Design three separate providedIn: 'root' services (NationalizeService, CountryService, UniversityService) inside the services folder

@src/app/features/customers/services/, rather than one enrichment service, since they will be cached differently.
The country list is called once then cached, university search must never cache since it will need be called as you type,

and Nationalize gets call per surname change.

Here are the urls that will be used for the services:
Country service - 'https://countries.dev/countries?fields=name,flag,flags,alpha2Code'
Nationalize service - 'https://api.nationalize.io'
University service - 'http://universities.hipolabs.com/search'

Lets start by getting the models right.
Show me the confirmed request/response shape for each API before we build the services around them.

Then add those models inside @src/app/core/models/enrichment.model.ts

Step 2 — Nationalize & Country services

Build NationalizeService.predict(name) returning Observable<{ predictions: {countryCode, probability}[], rateLimited: boolean }>.
On a 429 response due to the rate-limit you mentioned, return rateLimited: true with empty predictions instead of throwing.
On any other error, console.warn and return an empty, non-rate-limited result — this should never break the form.
Build CountryService.getCountries() backed by a single cached Observable<Country[]> ,

fetch it once on first subscription and keep it alive across component destroy/recreate with shareReplay({ bufferSize: 1, refCount: false }),
since the country list is static and shouldn't be re-fetched every time the enrichment panel remounts.

Map countries.dev's alpha2Code as the join key back to Nationalize's country_id.

Define the DTOs (NationalizeApiResponse, CountryApiEntry) separately from the clean view model types (Country, CountryPrediction).
On a failed countries.dev request, the service should log the error and emit an empty array rather than erroring the whole stream.
The form must remain usable even if enrichment data fails to load.

Step 3 — University service

Build UniversityService.search(countryFullName, query) calling the hipolabs endpoint.

Note it takes the full country name as a lowercase string, not a country code.
No caching here (unlike the country service), every keystroke is a genuinely new query.

Map each result to { name, website }, falling back to constructing a URL from domains[0] when web_pages is empty.

Catch errors and resolve to an empty array rather than propagating, a slow/failed university search shouldn't block form submission

Step 4 — Prediction pipeline in the enrichment panel

In the enrichment panel component @src/app/features/customers/enrichment/enrichment-panel.component.ts ,

wire the last-name control's valueChanges through:
trim -> distinctUntilChanged -> debounceTime(500ms)-> skip if under 2 chars -> call NationalizeService.predict ->

join the returned country codes against the cached country list -> sort by probability descending -> take the top 5.

Render predictions as a mat-chip-listbox here @src/app/features/customers/enrichment/enrichment-panel.component.html#10,

each chip showing the flag, country name, and probability as a percentage.

Show a spinner while loading and a "no predictions - search manually" message when the array comes back empty for a non-trivial surname.

Track the rate limited state separately from "no predictions" a 429 should show a distinct warning message telling the

user to try again shortly or pick manually, not be indistinguishable from a genuine empty result.

Step 5 — Manual override & confirmation

Below the predictions, always show a searchable autocomplete over the full cached country list so the

user can override the prediction entirely,
even if predictions loaded fine.

Selecting either a predicted chip or a manual autocomplete option should set the
same underlying countryControl value and show a "Confirmed: [flag] [name]" line.

If the user changes the confirmed country after already picking a university, clear the university selection,
a university tied to the old country shouldn't silently persist against a new one.

Step 6 — University search-as-you-type

Once a country is confirmed, reveal a university autocomplete.

Debounce the query at 400ms, require at least 2 characters, and re-run the search if the confirmed country itself changes.

Selecting a result stores { name, website } onto the customer form's university

control and shows a confirmation line with a link to the site and a clear button.
Keep this section visually subordinate to the country section (smaller, indented)

since it's a dependent step, don't let it render at all until a country is confirmed

Step 7 — Wiring into the customer form + final polish

Embed the enrichment panel in the add/edit customer form here @src/app/features/customers/customer-form.component.html#26-30,

passing direct references to the form's lastNameControl, countryControl, and universityControl rather than duplicating state
the panel writes directly onto the parent form's controls.

CountryCode and university must stay optional on the customer, a user should be

able to save the form without ever touching the enrichment panel.

Confirm the full loop end-to-end: type a strong-signal surname (e.g. "Marais") -> see ranked predictions with

flags -> confirm one -> search a university -> select it -> save the customer -> reopen the edit form and verify both values persisted correctly.

Bug fixes

I'm getting NG5002 - as expression only allowed on primary @if block in the enrichment panel template, here's the template, what's wrong?

Country flags are rendering as literal text like 'ZA' instead of the actual flag emoji,

screenshot attached. Is this a data problem or something else?

(Root cause was a Windows font limitation with emoji flags — resolved by switching the panel to render flags.svg image

URLs instead of the emoji character, and adding a flagUrl field to the Country model.)